Experiment No. 4

To implement a complete Transformer architecture from scratch using the PyTorch library, understanding all fundamental components including multi-head attention, positional encoding, encoder-decoder architecture, and feed-forward networks.

Install & Import liabraries

```
import torch
import torch.nn as nn
import math
```

Positional Encoding

```
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=5000):
        super(PositionalEncoding, self).__init__()

        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)

        div_term = torch.exp(torch.arange(0, d_model, 2).float() *
                             (-math.log(10000.0) / d_model))

        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)

        pe = pe.unsqueeze(0)
        self.register_buffer('pe', pe)

    def forward(self, x):
        x = x + self.pe[:, :x.size(1)]
        return x
```

Multi-Head Attention

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super(MultiHeadAttention, self).__init__()

        self.num_heads = num_heads
        self.d_model = d_model
        self.head_dim = d_model // num_heads

        assert d_model % num_heads == 0

        self.q_linear = nn.Linear(d_model, d_model)
        self.k_linear = nn.Linear(d_model, d_model)
        self.v_linear = nn.Linear(d_model, d_model)

        self.out = nn.Linear(d_model, d_model)

    def forward(self, q, k, v):
        batch_size = q.size(0)

        q = self.q_linear(q)
        k = self.k_linear(k)
        v = self.v_linear(v)

        q = q.view(batch_size, -1, self.num_heads, self.head_dim).transpose(1,2)
        k = k.view(batch_size, -1, self.num_heads, self.head_dim).transpose(1,2)
        v = v.view(batch_size, -1, self.num_heads, self.head_dim).transpose(1,2)

        scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(self.head_dim)
        attn = torch.softmax(scores, dim=-1)

        context = torch.matmul(attn, v)
        context = context.transpose(1,2).contiguous().view(batch_size, -1, self.d_model)

        output = self.out(context)
        return output
```

Feed Forward Network

```
class FeedForward(nn.Module):
    def __init__(self, d_model, d_ff):
        super(FeedForward, self).__init__()

        self.linear1 = nn.Linear(d_model, d_ff)
        self.linear2 = nn.Linear(d_ff, d_model)
        self.relu = nn.ReLU()

    def forward(self, x):
        return self.linear2(self.relu(self.linear1(x)))
```

Encoder Layer

```
class EncoderLayer(nn.Module):
    def __init__(self, d_model, num_heads, d_ff):
        super(EncoderLayer, self).__init__()

        self.attention = MultiHeadAttention(d_model, num_heads)
        self.norm1 = nn.LayerNorm(d_model)

        self.ff = FeedForward(d_model, d_ff)
        self.norm2 = nn.LayerNorm(d_model)

    def forward(self, x):
        attn_output = self.attention(x, x, x)
        x = self.norm1(x + attn_output)

        ff_output = self.ff(x)
        x = self.norm2(x + ff_output)

        return x
```

Full Transformer Encoder Model

```
class Transformer(nn.Module):
    def __init__(self, vocab_size, d_model, num_heads, d_ff, num_layers, max_len=100):
        super(Transformer, self).__init__()

        self.embedding = nn.Embedding(vocab_size, d_model)
        self.pos_encoding = PositionalEncoding(d_model, max_len)

        self.layers = nn.ModuleList(
            [EncoderLayer(d_model, num_heads, d_ff) for _ in range(num_layers)]
        )

        self.fc_out = nn.Linear(d_model, vocab_size)

    def forward(self, x):
        x = self.embedding(x)
        x = self.pos_encoding(x)

        for layer in self.layers:
            x = layer(x)

        output = self.fc_out(x)
        return output
```

Test the Model

```
vocab_size = 100
d_model = 64
num_heads = 8
d_ff = 256
num_layers = 2

model = Transformer(vocab_size, d_model, num_heads, d_ff, num_layers)

sample_input = torch.randint(0, vocab_size, (2, 10))  # (batch_size, sequence_length)

output = model(sample_input)

print("Input shape:", sample_input.shape)
print("Output shape:", output.shape)
```

```
Input shape: torch.Size([2, 10])
Output shape: torch.Size([2, 10, 100])
```